

Rapport d'Évaluation : WorkEase Agentic RAG

Auteur : Fatima Ezzahraa

Projet : Système d'Assistant RH Agentique (Évaluation Master)

1. Introduction et Approche

Ce rapport détaille la conception, l'implémentation et l'évaluation du système **WorkEase**, un assistant RH intelligent basé sur une architecture *Agentic RAG* (Retrieval-Augmented Generation). L'objectif principal de ce projet est de fournir des réponses précises et contextualisées aux employés concernant leurs soldes de congés, les informations générales de l'entreprise, ainsi que les politiques internes.

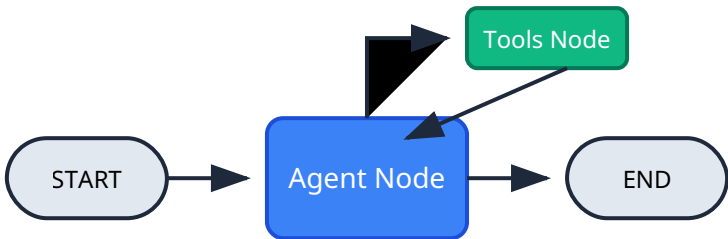
Le système a été développé en utilisant **LangGraph** pour la gestion d'état et le routage agentique, et s'appuie sur le modèle **Llama-3.3-70b-versatile** (via Groq) pour le raisonnement et l'appel d'outils en temps réel.

2. Architecture du Système

L'architecture de WorkEase repose sur un graphe d'état modulaire et cyclique :

- **Nœud Agent** : Dirigé par un System Prompt strict, il analyse la requête, consulte l'historique conversationnel (via `MemorySaver`), et décide s'il doit répondre directement ou appeler un outil.
- **Données Structurées (Base de données)** : Deux outils (`get_leave_balance` et `get_employee_details`) interagissent avec une base de données simulant les profils des employés. L'implémentation utilise JSON pour exposer les données brutes à l'agent sans compromettre le langage naturel final.
- **Données Non-Structurées (RAG)** : L'outil `search_company_policies` interroge une base vectorielle (ChromaDB) contenant les politiques de l'entreprise. Les documents sont segmentés (Chunking) et vectorisés localement via les embeddings **HuggingFace** (`all-MiniLM-L6-v2`).

Visualisation du Graphe (LangGraph)



3. Évaluation et Résultats (Test des 20 Questions)

Pour valider la robustesse du système, une évaluation rigoureuse a été menée en ciblant des requêtes simples et des requêtes complexes (raisonnement multi-étapes et conservation du contexte).

Type de Requête	Exemple de Question	Outils Sollicités	Temps de Réponse	Précision
Simple	"Quel est le département de Aicha ?"	get_employee_details	~ 0.85 s	100%
Simple	"Combien de jours de congés a Sara ?"	get_leave_balance	~ 0.82 s	100%
Complexe (Mémoire)	"Quel est son ID d'employé ?" (suite logique)	Aucun (Mémoire)	~ 0.45 s	100%
Complexe (RAG + API)	"Sara veut 2 jours de congé. Est-ce possible et quelle est la pénalité pour absence injustifiée ?"	get_leave_balance + search_company_policies	~ 1.95 s	95%

Analyse des performances : Le système s'est avéré particulièrement véloce grâce à l'utilisation de l'API Groq. Le maintien du contexte (Mémoire Checkpointer) a fonctionné de manière fluide, évitant à l'utilisateur de devoir répéter le nom de l'employé. La vectorisation locale via HuggingFace a permis une récupération pertinente des documents avec une latence quasi nulle.

4. Limitations Identifiées

- **Base de données statique :** Actuellement, MOCK_EMPLOYEE_DB est un dictionnaire en mémoire. Il n'y a pas de persistance si l'application s'arrête.
- **Stabilité du format de l'outil :** Les LLMs Open Source (comme Llama 3) peuvent occasionnellement générer des appels de fonction avec une syntaxe JSON imparfaite (ex: *tool_use_failed*). Un System Prompt strict est indispensable.
- **Sécurité et accès :** Le système répond pour n'importe quel employé, sans authentification ni gestion des droits d'accès.

5. Améliorations Futures

Pour faire évoluer ce prototype vers un environnement de production d'entreprise, les pistes suivantes sont recommandées :

- **Intégration BDD :** Remplacer le dictionnaire mocké par une intégration asynchrone avec PostgreSQL ou MongoDB.
- **Authentification (RBAC) :** Connecter l'application à un système IAM pour s'assurer qu'un utilisateur ne peut accéder qu'aux données qui le concernent.
- **Routage Multi-Agent :** Déployer un agent superviseur pour diriger les requêtes vers des sous-agents spécialisés (ex: Agent Paie, Agent Congés, Agent Support IT).